Basic (your first working tool) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `first_tool_call_practice.py`: this is the exact primitive every agent in this entire curriculum is built from — a model deciding to call a function instead of answering directly. Seeing it work once, standalone, before combining it with anything else, is what makes the Build Task's `run_with_tools()` obvious instead of magic. **If not:** the first time you'd see a tool call requested and executed would be inside a bigger harness already juggling 2-3 tools, making it harder to tell which part is "the round trip" and which part is everything else.

Basic Version

Approach 1 — the direct way

```
# first_tool_call_practice.py
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def add(a: int, b: int) -> int:
    """Add two integers together and return the sum."""
    return a + b

model_with_tools = ChatOpenAI().bind_tools([add])

response = model_with_tools.invoke("What's 5 + 7?")
print("Requested tool calls:", response.tool_calls)

for call in response.tool_calls:
    if call["name"] == "add":
        result = add.invoke(call["args"])
        print("Real result:", result)
```

Expected output:

```
Requested tool calls: [{'name': 'add', 'args': {'a': 5, 'b': 7},
    'id': 'call_abc123', 'type': 'tool_call'}]
```

```
Real result: 12
```

This version works correctly. It doesn't handle the case where the model calls something other than `add`, or calls nothing at all — fine for a first working version, worth fixing next.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — checked, and wrapped in real functions

```
# first_tool_call_practice.py
from langchain_core.runnables import Runnable
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI


@tool
def add(a: int, b: int) -> int:
    """Add two integers together and return the sum."""
    return a + b


def get_model_with_tools() -> Runnable:
    # how: .bind_tools() returns a new Runnable, not a ChatOpenAI
    return ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools([add])


def main() -> None:
    model_with_tools = get_model_with_tools()
    response = model_with_tools.invoke("What's 5 + 7?")

    if not response.tool_calls:
        # when: even an obvious math question, the model can still just
        # answer in text instead of calling a tool — always check first.
        print("The model answered directly, without calling a tool.")
        print(response.content)
        return

    for call in response.tool_calls:
        if call["name"] == "add":
            # how: .invoke() on the tool itself, not a plain function call —
            # @tool wraps add into a Runnable too.
            result = add.invoke(call["args"])
            print(f"Called add({call['args']}) -> {result}")
        else:
            print(f"Unexpected tool requested: {call['name']}")
```

```
if __name__ == "__main__":
    main()
```

Expected output:

```
Called add({'a': 5, 'b': 7}) -> 12
```

Difference from Basic: checking `if not response.tool_calls` handles the case where the model just answers directly instead of calling anything — a real possibility even for an obvious math question, and one your code needs to not crash on. Checking `call["name"]` before running it also matters the moment you register a second tool, since you can't assume the first (or only) requested call is the one you expected.

Which one should you actually write? For this exercise, Intermediate's checked version is enough. A real round trip — looping over every call in a response, tagging each result back with `ToolMessage(tool_call_id=call["id"], ...)`, and calling the model a second time for its actual final answer — is worth knowing this exercise stops short of; Doc07's agent loop is exactly that pattern, run repeatedly. Forcing a specific tool with `tool_choice="add"` is worth reaching for only when you, the developer, already know which tool must run next — not for an exercise where the model deciding is the whole point.